How My Tests Work

demoblaze automation — a plain-English walkthrough of both test suites
Repository: BlazaMeterWebsite-Automation-Seleniu-with-Python- | Branch: POM-Pytest

1. The big picture

You now have **two separate sets of tests** in one project. They test the same website in two completely different ways.

Suite	What it does	How it feels
UI tests (Selenium)	Opens a real Firefox window and clicks through the website like a human being would.	Slow (about a minute), realistic, breaks easily if the page design changes.
API tests (requests)	Talks straight to the server behind the website. No browser at all.	Fast (about 16 seconds for 14 tests), stable, but never sees the actual page.

Think of it like testing a restaurant. The **UI test** walks in, sits at a table, reads the menu and orders out loud. The **API test** phones the kitchen directly and asks "do you have the Samsung Galaxy S6, and how much is it?" Both are useful. They catch different problems.

> **Important:** Selenium *cannot* do API testing. Selenium's only job is to control a browser. To test an API you need a different tool — here it is a Python library called `requests`. The two simply live in the same project and run with the same command.

2. The UI test, step by step

When you run `pytest -m ui`, exactly one test runs: `test_purchase_flow`. It performs the whole "buy something" journey in one go. Here is every step, in order.

1. **Firefox opens** and goes to demoblaze.com. (done by the `Browser` fixture in `conftest.py`)
2. **Clicks "Sign up"** in the top menu, types the username `Shahrukh` and password `12345`, and clicks the Sign up button.
3. **A popup appears.** The test reads its message and clicks OK. (In practice this says "This user already exist." — the account was created long ago.)
4. **Closes** the sign-up box.
5. **Clicks "Log in"** in the top menu.
6. **Types the username** into the login box.
7. **Types the password** into the login box.
8. **Clicks the "Log in" button** and waits for the page to update.

9. **Clicks the product** “Samsung galaxy s6” on the home page.
10. **Clicks “Add to cart”** on the product page.
11. **A popup says the product was added.** The test clicks OK.
12. **Clicks “Cart”** in the top menu to open the basket.
13. **Clicks “Place Order”**.
14. **Types the name** `shahrukh` into the order form.
15. **Types the card number** `4111 1111 1111 1111` into the order form.
16. **Clicks “Purchase”**.
17. **Clicks “OK”** on the confirmation message. The order is complete.

How the code is organised

Those 17 steps are not written in one long file. They are split up, which is called the **Page Object Model**:

```
test_purchase_flow.py   the test    ->  "run the purchase journey"
        |
Test/Action.py          the recipe  ->  calls the 17 steps in the right order
        |
Pages/*.py              the steps   ->  each file knows how to click one part of the site
```

The benefit: if demoblaze ever renames its “Add to cart” button, you change **one line in one file** (`Pages/Test_AddToCart.py`) instead of hunting through every test.

> **Something to know about this test:** it does not actually *check* anything. There is no line saying “the price should be $360” or “the confirmation should appear.” The test only fails if a button is missing. So it proves the journey is *clickable* — it does not yet prove the shop is *correct*. Adding a few `assert` lines is the single biggest improvement you could make.

3. The API tests, step by step

When you run `pytest -m api`, 14 tests run in about 16 seconds and no browser opens. Here is what happens, in order.

Step 1 — Log in once

Before any test runs, the suite sends the username and password to `https://api.demoblaze.com/login` and gets back a **token** — a short code that proves "I am Shahrukh". Every later test reuses that same token instead of logging in again.

Step 2 — Check the product catalogue (6 tests)

- Ask for **all products** and check every one has an id, title, price, category, image and description — and that the price is actually a number, not text.
- Ask for **only phones**, then **only notebooks**, then **only monitors**, and check nothing from another category sneaks into the answer.
- Ask for a **category that does not exist** and check the server politely returns an empty list instead of crashing.
- Ask for **one specific product** (id 1) and check it really is the Samsung galaxy s6 at $360.

Step 3 — Check logging in (4 tests)

- Log in with the **right password** → a token comes back.
- Log in with the **wrong password** → the server must say exactly "Wrong password."
- Log in as a **user who does not exist** → must say exactly "User does not exist."
- Sign up with a username that is **already taken** → must say "This user already exist."

That last one is deliberate. demoblaze is a public website shared by thousands of learners, and it has no way to delete an account. If the tests created a new user on every run, we would be leaving rubbish on someone else's server forever. So we test sign-up using the account that *already* exists — real coverage, nothing created.

Step 4 — Check the shopping cart (4 tests)

- **Add an item** to the cart, then look at the cart and confirm it is there.
- **Use a fake token** and confirm the server refuses with "Bad parameter, token malformed."
- **Delete the item** and confirm the cart is empty again.

Every cart test cleans up after itself. Even if a test fails halfway, the item is still removed — so your cart never fills up with leftovers from old runs.

4. Two surprises we found in demoblaze

Surprise 1: the server says "200 OK" even when things go wrong

Normally a website answers with a number: **200** means "fine", **404** means "not found", **401** means "you're not allowed". demoblaze answers **200 for absolutely everything** — even for a wrong password.

```
Wrong password       -> 200 OK   {"errorMessage": "Wrong password."}
User does not exist  -> 200 OK   {"errorMessage": "User does not exist."}
Complete nonsense    -> 200 OK   {"Items": []}
```

This matters a lot. Most tutorials write `assert response.status_code == 200` and think they have tested something. On this site that line **passes even when the request failed**. So our tests ignore the number and read the actual message instead. That logic lives in one file, `API/asserts.py`, so it is written once rather than fourteen times.

Surprise 2: the delete button does not delete

demoblaze has an endpoint called `/deletecart`. It replies **"Item deleted."** — and the item is still sitting in the cart. We checked repeatedly; it is not a delay. A different endpoint, `/deleteitem`, is the one that actually works.

```
/deletecart -> "Item deleted."   then check the cart -> item is STILL THERE
/deleteitem -> "Item deleted."   then check the cart -> cart is empty
```

> **You found a genuine bug in the website you are testing.** That is exactly what testing is for. The test `test_deletecart_removes_the_item` is marked `xfail`, which means "we expect this to fail". It shows as **XFAIL** in the report instead of turning your run red. And if demoblaze ever fixes it, the test will loudly tell you so you can remove the marker.

5. What every file does

File	Its job, in one line
`conftest.py`	Opens and closes Firefox; saves a screenshot when a UI test fails.
`pytest.ini`	Settings: which files are tests, and the `api` / `ui` labels.
`test_purchase_flow.py`	The one UI test.
`Test/Action.py`	The recipe — calls the 17 UI steps in order.
`Pages/*.py`	One file per part of the website (login, cart, product…).
`API/config.py`	The website address, username and password.
`API/client.py`	Knows how to talk to the server — the API version of the Pages files.
`API/asserts.py`	Decides whether an answer counts as success or failure.
`API/conftest.py`	Logs in once; adds and removes cart items around each test.
`API/test_*.py`	The 14 API tests themselves.
`reports/`	HTML reports you can open in a browser.
`screenshots/`	Pictures saved automatically when a UI test fails.

6. How to run everything

```
pip install -r requirements.txt      install the tools (once)

pytest -m api                        just the API tests   (fast, no browser)
pytest -m ui                         just the UI test     (opens Firefox)
pytest                               both

pytest -m api --html=reports/api_report.html --self-contained-html
                                     ...and save a report you can open
```

The `-m` means “only run tests with this label”. The labels come from the `@pytest.mark.api` and `@pytest.mark.ui` lines in the test files.

7. Reading the result

```
13 passed, 1 xfailed, 1 deselected in 15.75s
```

Word	What it means
passed	The test checked something and it was correct.
failed	The test checked something and it was wrong. Read the message underneath.
xfailed	Failed *on purpose* — a known bug we already know about. Not your problem.
xpassed	A known bug got fixed! Go and remove the `xfail` marker.
deselected	Skipped because of the label — e.g. the UI test when you ran `-m api`.
error	The test never even started — usually the setup broke (browser, network).

8. Honest list of things to improve next

1. **Add assertions to the UI test.** Right now it clicks through the site but never checks that anything is correct. Even three checks — the product name, the price, the confirmation message — would transform it.
2. **Replace `time.sleep` with proper waits.** The UI flow contains about **64 seconds** of fixed sleeping. `WebDriverWait` waits only as long as it actually needs to — usually a fraction of a second — and is also more reliable on a slow day.
3. **Rename the files in `Pages/`.** They are called `Test_Login.py` with `class TestLogin`, so pytest used to mistake all 17 page-object methods for real tests and run them in the wrong order. It is currently patched with a setting; renaming them to `login_page.py` / `class LoginPage` is the proper fix.
4. **Move the `time.sleep(7)` in `Test_ClickProduct.py` inside the method.** It sits directly under `class TestClickProduct:`, so it runs once when Python loads the file — not when the test runs. It is delaying the wrong thing.
5. **Use the API to speed up the UI test.** Once you are comfortable, you can log in through the API and hand the token to the browser, skipping steps 2–8 entirely. That is the normal next step in a real framework.
